Machine Learning Mini-Project 4

Regularization, Optimization, CNNs, LSTMs, and Word Embeddings

Machine Learning

July 11, 2026

Overview and Learning Objectives

This miniproject deepens your understanding of modern deep learning. You will:

- Compare regularization strategies (early stopping vs. dropout) in a regression task.
- Contrast optimization algorithms (SGD vs. Adam) on MNIST digit classification.
- Build and evaluate a Convolutional Neural Network (CNN) on CIFAR10 images.
- Build and evaluate a Long ShortTerm Memory (LSTM) network for IMDB sentiment analysis.
- Implement the Continuous BagofWords (CBOW) model and visualize word embeddings.

The emphasis is on practical implementation, rigorous evaluation (accuracy, confusion matrix, ROC curves, learning curves), and critical analysis of results.

Datasets

- **Synthetic regression** generated from $f(x_1, x_2) = x_1^3 - x_1^2 + 2x_2^2 + 3x_1x_2 + 5$ with noise.
- **MNIST** 28×28 grayscale images of digits 09.
- **CIFAR10** 32×32 colour images in 10 classes.
- **IMDB** 50 000 movie reviews for binary sentiment classification.
- **Custom text** a 1000word excerpt from the internet on a single topic (choose your own, e.g., a Wikipedia article).

Part 1: Deep Feedforward Networks and Backpropagation

Consider a fully connected neural network with the architecture: Input (2 nodes) → Hidden Layer (3 ReLU nodes) → Hidden Layer (2 ReLU nodes) → Output (1 node, linear activation). The input features are x_1, x_2 and the output is a scalar y.

a) Write the forward pass equations for all layers. Define all weight matrices and bias vectors explicitly.

b) Derive the backpropagation equations for updating the weights of the first layer using the squared loss function:

$$\mathcal{L} = \frac{1}{2}(y - \hat{y})^2$$

c) Using the following data point: $x = [1, -1]^\top$, $y = 2$, and random initialization:

$$W^{[1]} = \begin{bmatrix} 0.1 & -0.2 \\ 0.05 & 0.3 \\ -0.3 & 0.1 \end{bmatrix}, \quad b^{[1]} = [0, 0, 0]^\top, \quad W^{[2]} = \begin{bmatrix} 0.1 & 0.2 & -0.4 \\ 0.01 & -0.3 & 0.2 \end{bmatrix}, \quad b^{[2]} = [0, 0]^\top,$$

$$W^{[3]} = [0.4, -0.1, 0.2], \quad b^{[3]} = 0,$$

perform a single forward and backward pass and update the weights using learning rate $\eta = 0.1$.

Part 2: Regularization for Deep Learning

You are training a deep neural network and want to prevent overfitting using different regularization techniques.

a) Explain how L2 regularization affects the weight updates in gradient descent. Show how the loss function changes and how the gradient is modified.

b) Consider a scenario where dropout is applied to a hidden layer. Describe how dropout works during training and how it modifies the network during inference.

c) You are training a model on a small dataset. Compare the expected behavior of the following methods: (i) Early stopping, (ii) Dropout, and (iii) L2 regularization. When might one outperform the others?

d) Consider the function

$$f(x_1, x_2) \;=\; x_1^3 \;-\; x_1^2 \;+\; 2\,x_2^2 \;+\; 3\,x_1 x_2 \;+\; 5.$$

You will generate noisy samples from this function, train a multilayer perceptron (MLP) regressor, and compare the effects of *early stopping* versus *dropout* on both training time and test accuracy.

2.1 Data Generation

(a) Draw $N = 1000$ independent samples $\{(x_1^{(i)}, x_2^{(i)})\}_{i=1}^N$, uniformly from the square $[-10, 10] \times [-10, 10]$.

(b) Compute the noiseless targets $y^{(i)} = f\big(x_1^{(i)}, x_2^{(i)}\big)$.

(c) Add additive noise: for each i, sample $\varepsilon^{(i)} \sim \mathcal{U}(-1, 1)$ and set

$$\tilde{y}^{(i)} \;=\; y^{(i)} \;+\; \varepsilon^{(i)}.$$

(d) Split the dataset into a training set (80%) and a test set (20%).

2.2 Model Architecture

(a) Define an MLP with the following structure:

- Input layer of size 2.
- Two hidden layers, each with 10 neurons and ReLU activation.
- A single linear output neuron for regression.

(b) Choose mean squared error (MSE) as the loss function.

(c) Use the Adam optimizer with default parameters.

2.3 Training and Evaluation

(a) Train the MLP for up to 200 epochs on the training set, recording:

- Training loss (MSE) per epoch.
- Validation loss (MSE) per epoch, using 10% of the training set as a validation split.

(b) After training, compute the test set MSE on the heldout 20% of samples.

(c) Plot:

- Training and validation loss curves vs. epoch.
- True vs. predicted outputs on the test set (scatter plot).

2.4 Regularization Strategies

(a) *Early Stopping:*

- Retrain the model from scratch, this time applying early stopping with patience of 10 epochs.
- Report the epoch at which training stopped, the training time, and the final test MSE.

(b) *Dropout:*

- Retrain the original 2layer MLP, but add a Dropout layer (drop probability $p = 0.5$) after each hidden layer.
- Train for the full 200 epochs (no early stopping).
- Report total training time and final test MSE.

(c) **Comparison:**

- Compare early stopping vs. dropout in terms of:

  i) Total training time

  ii) Test MSE

- Discuss which strategy you would recommend for this regression task and why.

Part 3: Optimization for Training Deep Networks

Training deep networks can be challenging due to optimization difficulties such as vanishing gradients, poor initialization, and slow convergence.

a) Compare SGD, Momentum, and Adam optimizers in terms of how they update the weights. Provide the update rules for each.

b) Suppose your deep network suffers from vanishing gradients. Explain how this problem arises and suggest two strategies to mitigate it.

3.1 Data Preparation

Load MNIST, normalise pixel values to $[0, 1]$, and convert labels to onehot encoding.

3.2 Model

Build a 3layer MLP:

- Input: 784
- Hidden 1: 128, ReLU
- Hidden 2: 64, ReLU
- Output: 10, softmax

3.3 Compare Optimizers

Train two identical instances of the same model using:

- **SGD** with learning rate 0.01 and momentum 0.9.
- **Adam** with default learning rate.

For each, run for 20 epochs, record training/validation accuracy and loss per epoch. Plot the learning curves on the same graph for direct comparison. Report final test accuracy and comment on convergence speed and stability.

Part 4: CNN vs. LSTM on Benchmark Datasets

In this problem you will:

- Train a Convolutional Neural Network (CNN) on CIFAR-10 (image classification).
- Train a Long Short-Term Memory network (LSTM) on IMDB (text sentiment).
- Compare training behavior, performance, and resource usage of both models.

4.1 Experiment 1: CNN on CIFAR-10

a. **Data Preparation**

- Load CIFAR-10: 50 000 training images, 10 000 test images, 10 classes.
- Normalize pixel values to $[0, 1]$.

- One-hot encode labels.
- (Optional) Apply simple augmentations: random flips, shifts.

b. **Model Architecture**

- Input: $32 \times 32 \times 3$.
- *Block 1:*
  - Conv2D (32 filters, 3×3, ReLU)
  - Conv2D (64 filters, 3×3, ReLU)
  - MaxPooling (2×2), Dropout ($p = 0.25$)
- Flatten $\rightarrow$ Dense(512, ReLU) $\rightarrow$ Dropout($p = 0.5$).
- Output Dense(10, softmax).

c. **Training**

- Loss: Categorical cross-entropy.
- Optimizer: Adam.
- Batch size: 64. Epochs: 50.
- Validation split: 10% of training data.
- Record training/validation loss & accuracy each epoch.

d. **Evaluation**

- Compute test accuracy.
- Display confusion matrix.
- Plot ROC curves (one-vs-rest).
- Show a few misclassified images (true vs. predicted labels).

4.2 Experiment 2: LSTM on IMDB Sentiment

a. **Data Preparation**

Before feeding text into an LSTM, we must convert it into a numerical form of fixed length:

- *Load the IMDB dataset:*
  - 25 000 movie reviews for training, and 25 000 for testing.
  - Each review is already labeled as positive or negative.
- *Tokenization and Vocabulary Selection:*
  - Build a word-to-index mapping using only the **20 000** most frequent words.
  - All less frequent words are replaced by a special <UNK> token.
- *Padding and Truncation:*

  - Choose a fixed sequence length of **200** tokens for each review.
  - **Padding:** If a review has fewer than 200 words, prepend zeros (or a <PAD> index) until its length is 200.
  - **Truncation:** If a review has more than 200 words, discard words beyond the first 200.
- *Result:*
  - Each review becomes a vector in $\{0, \ldots, 20000\}^{200}$.
  - This uniform shape enables efficient batch processing in the LSTM.

b. **Model Architecture** We now define a simple LSTM-based classifier that can learn from these fixed-length sequences:

- *Input Layer:*
  - Accepts a sequence of 200 integer word indices.
- *Embedding Layer:*
  - Transforms each word index into a 128-dimensional dense vector.
  - **Parameters:** vocabulary size = 20 000; embedding dimension = 128.
  - Learns word representations specialized for the sentiment task.
- *LSTM Layer:*
  - Contains 128 LSTM units (or cells).
  - Processes the sequence one time-step (word) at a time, maintaining an internal state that captures context.
  - Outputs a single 128-dimensional vector summarizing the entire review.
- *Fully Connected + Dropout:*
  - Dense layer with 64 neurons and ReLU activation: introduces additional nonlinear modeling capacity.
  - Dropout with probability $p = 0.5$: randomly disables half of the neurons during training to reduce overfitting.
- *Output Layer:*
  - Dense layer with 1 neuron and `sigmoid` activation.
  - Produces a probability in $[0, 1]$ indicating the reviews predicted positivity.

c. **Training**

- Loss: Binary cross-entropy.
- Optimizer: Adam.
- Batch size: 64. Epochs: 20.
- Validation split: 10%.

- Record training/validation loss & accuracy each epoch.

d. **Evaluation**

- Compute test accuracy, precision, recall, F1-score.
- Plot learning curves (loss & accuracy vs. epoch).
- Show examples of misclassified reviews and discuss.

Part 5: Continuous bag of words (CBOW) Word Embedding

In this question, you will implement a Continuous Bag-of-Words (CBOW) model using TensorFlow. You will learn how to extract word embeddings in different dimensions and visualize them in 2D. Use a simple 1000 word text from internet which discuss a specific topic.

Direct 2D Embedding

Implement a CBOW model that directly learns 2-dimensional word embeddings.

5.1 Model definition: Define a CBOW architecture in TensorFlow with

- Input: context window of size $w = 2$ (two words before and after),
- Embedding layer: dimension $d = 2$,
- Output: softmax over the vocabulary.

5.2 Training:

- Use cross-entropy loss and the Adam optimizer.
- Train for at least 300 epochs (or until convergence).

5.3 Visualization:

(a) Extract the learned embedding matrix $E \in \mathbb{R}^{V \times 2}$, where V is the vocabulary size.

(b) Create a scatter plot of the 2D points, labeling each point with its corresponding word.

(c) Discuss any clusters or semantic patterns you observe.

5.4 10D Embedding with PCA Reduction

Now you will learn embeddings in a higher-dimensional space and then apply Principal Component Analysis (PCA) to project them into 2D.

a) **Model definition:** Modify your CBOW implementation to use an embedding dimension $d=10$.

b) **Training:** Train the model as in Problem 1 until the loss stabilizes.

c) **Extraction:** Extract the embedding matrix $E_{10} \in \mathbb{R}^{V \times 10}$.

d) **PCA projection:**

(a) Perform PCA on E_{10} to obtain the first two principal components.

(b) Project each 10D embedding onto this 2D subspace, obtaining $E_{\text{PCA}} \in \mathbb{R}^{V \times 2}$.

e) **Visualization and analysis:**

- Plot the PCA-reduced embeddings as in previous subsection.
- Compare the plot shapes and clusters with those from the direct 2D model.
- Provide explanations for any similarities or differences.

Comprehensive Discussion

Your report must include a final section that addresses:

- How early stopping and dropout influence the biasvariance tradeoff in the regression task.
- Why Adam often converges faster than SGD, and any tradeoffs observed in the MNIST experiment.
- The strengths of CNNs for image data and LSTMs for sequential text; how do their inductive biases match the data?
- The impact of embedding dimensionality on the quality of word representations; strengths and limitations of CBOW.
- Practical considerations: training time, hardware requirements, and ease of implementation for each method.

Deliverables

1. Handwritten Report (Maximum 10 Pages)

The report must be handwritten (on paper or tablet with stylus), scanned as a single PDF.

2. Video Explanation (7 Minutes)

Screencapture video with narration covering:

- Quick notebook walkthrough.
- Demonstration of key code sections (e.g., dropout vs. early stopping, CNN training, CBOW).
- Highlighting of the most important plots and results.
- Summary of findings and practical recommendations.

Upload to a platform (YouTube unlisted, Google Drive, etc.) and include the link in `video_link.txt`.

3. Jupyter Notebook

Submit a single `.ipynb` notebook containing all code, outputs, and markdown explanations, separated into clearly labelled sections for Parts 15.

Submission

Zip the following into `LastName_FirstName_ML_MiniProject4.zip`:

- Handwritten report PDF
- Jupyter notebook
- `video_link.txt`

Submit via the course portal by **8 Mordad 1405**.